

CS 5330 Project 1: Video Special Effects

Akashdeep Gangatkar Madhusudhan

September 30, 2026

1 Project Overview

In this project I built a set of C++ programs using OpenCV. They read and display still images, capture live video from a webcam, and apply special effects to each frame in real time. The first program (`imgDisplay`) loads an image from disk and shows it in a window until a key is pressed. The second program (`vidDisplay`) opens the built-in camera, shows the video stream frame by frame, and responds to keyboard commands. Pressing `q` quits, `s` saves the current frame to disk, and each other key toggles an effect on or off. The project covers the basics of image representation in OpenCV (the `cv::Mat` type, BGR colour channels, single- and multi-channel images), the capture–process–display loop, and pixel-level image processing.

2 Tasks

Task 1: Read and display an image

`imgDisplay` reads an image from the `data` folder and shows it in a window. It then loops, checking for key presses, and quits when the user types `q`.

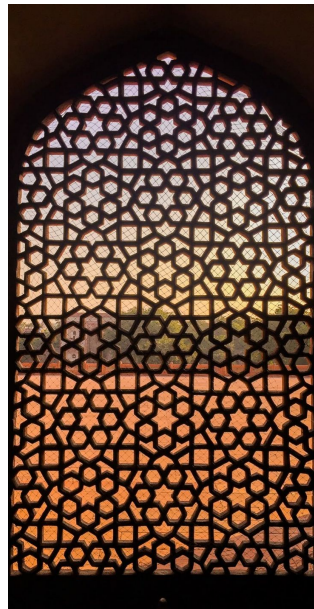


Figure 1: Image displayed by `imgDisplay`.

Task 2: Display live video

`vidDisplay` opens a video channel to the default webcam, creates a display window, and runs a loop. Each pass captures a new frame, displays it, and checks for a key press. Later tasks add more effects to this program, each switched on by its own key. `q` ends the program and `s` saves the current frame as a JPEG in the `data` folder.



Figure 2: A colour frame from the live webcam stream, saved with the `s` key.

Task 3: Greyscale live video

Pressing `g` toggles greyscale mode on and off. The program remembers the current mode, so every later frame is shown in that mode until `g` is pressed again. In greyscale mode each frame is converted with OpenCV's `cvtColor` using the `COLOR_BGR2GRAY` code. According to the OpenCV documentation (*Color Conversions*, `imgproc` module), this conversion computes a weighted sum of the three colour channels:

$$Y = 0.299 R + 0.587 G + 0.114 B$$

These are the standard ITU-R BT.601 luma weights. The weights reflect how sensitive the human eye is to each colour, green most of all and blue least. The output is a single-channel image in which each pixel's brightness approximates perceived luminance.



(a) Original colour frame



(b) OpenCV greyscale (`cvtColor`)

Figure 3: The original colour frame (left) and the `cvtColor` greyscale version of the same frame (right).

Task 4: Custom greyscale

Pressing `h` toggles a custom greyscale filter, which I wrote as a separate function in `filter.cpp`. The program visits every pixel and sets its grey value to the *largest* of its three colour values:

$$Y = \max(R, G, B)$$

That value is copied into all three channels, so the output stays a three-channel image of the same size and type as the input. It is the same as the Value channel of the HSV colour space.

I chose this method because `cvtColor` weights the channels very unevenly: blue counts for only 11% and green for almost 59%. As a result, strongly coloured blue and red objects look much darker than they appear to the eye. Taking the brightest channel treats all three colours equally. A pixel is judged by how much light it has in its strongest colour, whatever that colour is.

Differences from the default greyscale.

- **Brighter overall.** The maximum of three numbers is never smaller than any weighted average of them, so every pixel is at least as bright as in the `cvtColor` version.
- **Saturated colours become bright.** A pure blue pixel $(R, G, B) = (0, 0, 255)$ becomes 29 with `cvtColor` but 255 with the custom filter. Pure red goes from 76 to 255.
- **Less separation between hues.** Pure red, green, and blue all map to the same value (255), so the custom version loses some of the contrast between different colours.
- **Neutral areas are unchanged.** Pixels that are already grey (white, black, or equal R, G, B) look the same in both versions.



Figure 4: The same frame in colour, with OpenCV's greyscale, and with the custom max-channel greyscale. Strongly coloured regions look much brighter in (c).

Task 5: Sepia tone

Pressing `p` toggles a sepia-tone filter, which gives the video the warm brown look of an antique photograph. Each output channel is a weighted mix of all three input channels:

$$R' = 0.393 R + 0.769 G + 0.189 B$$

$$G' = 0.349 R + 0.686 G + 0.168 B$$

$$B' = 0.272 R + 0.534 G + 0.131 B$$

The red row has the largest weights and the blue row the smallest, which gives the image its yellow-brown tint. The red and green rows sum to more than 1, so bright pixels can exceed 255. Each

result is therefore capped at 255 before it is stored, to stop the value from wrapping around to a dark colour.

Using the original RGB values. All three new values depend on all three *original* values. If the new red were written back into the pixel before the new green and blue were computed, those would use the modified red and the colours would be wrong. I avoided this in two ways:

1. For each pixel, the original R , G , and B are first copied into separate local variables. All three new values are computed from those copies before anything is written.
2. The results are written into a separate output image. The input frame is never modified, so every pixel is always read in its original form.

Vignette. Pressing `v` applies sepia plus a vignette that darkens the image towards its edges. Each pixel's brightness is scaled by $1 - 0.7d^2$, where d is the pixel's distance from the image centre, normalised so the corners are at $d = 1$. The centre keeps its full brightness and the corners keep 30%.



Figure 5: The sepia-tone filter applied to a live frame, without (b) and with (c) the vignette effect.

Task 6: 5×5 Gaussian blur

Pressing `b` toggles a 5×5 blur. The filter uses an integer approximation of a Gaussian kernel whose entries sum to 100:

$$\frac{1}{100} \begin{bmatrix} 1 & 2 & 4 & 2 & 1 \\ 2 & 4 & 8 & 4 & 2 \\ 4 & 8 & 16 & 8 & 4 \\ 2 & 4 & 8 & 4 & 2 \\ 1 & 2 & 4 & 2 & 1 \end{bmatrix}$$

Each colour channel is blurred separately. The outer two rows and columns are copied from the original image, because the full 5×5 neighbourhood does not fit there.

I implemented two versions:

- **blur5x5_1 (naive):** for every pixel and channel, it multiplies the 25 neighbours by the kernel and sums them. Every pixel is accessed with `at<>`, which recomputes the pixel's memory address on each access.
- **blur5x5_2 (separable):** the 5×5 kernel is the outer product of `[1 2 4 2 1]` with itself. The blur can therefore be done as a horizontal pass followed by a vertical pass, which takes 10 multiplications per pixel instead of 25. The horizontal sums are kept in a 16-bit intermediate image, since they can reach $10 \times 255 = 2550$. Pixels are accessed through row pointers instead of `at<>`. The live video uses this version.

Timing. Both functions were timed on the same 736×1431 image, averaged over 10 runs per function and repeated three times. The measurements were made on an Apple M3 Pro with a debug build (-g, no compiler optimisation).

Run	blur5x5_1 (ms)	blur5x5_2 (ms)	Speed-up
1	478.9	73.4	6.5×
2	459.2	73.6	6.2×
3	459.1	72.5	6.3×
Mean	465.7	73.2	6.4×

Table 1: Average time per image for the two blur implementations.

The separable, pointer-based version is about 6.4× faster. The two passes cut the work per pixel from 25 multiply-adds to 10. Row pointers remove the address calculation that `at<>` performs on every access, and they read memory in order along each row.

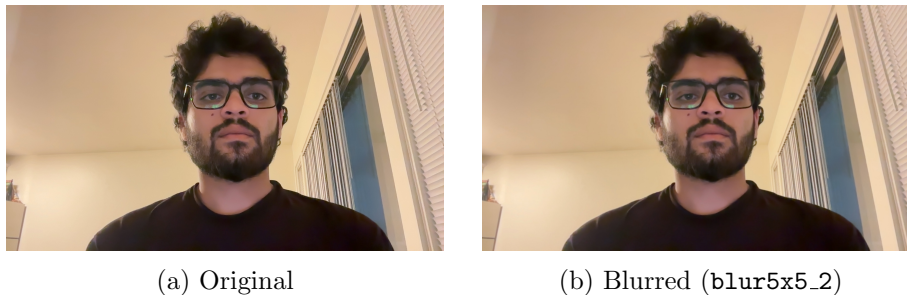


Figure 6: The test image before and after the 5×5 Gaussian blur. Fine detail and edges are softened.

Task 7: 3×3 Sobel X and Y filters

Pressing `x` or `y` shows the horizontal or vertical Sobel edge response of each colour channel. The kernels are:

$$S_X = \frac{1}{4} \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \quad S_Y = \frac{1}{4} \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$

Sobel X is positive where the image gets brighter to the right, so it responds to *vertical* edges. Sobel Y is positive where the image gets brighter upwards, so it responds to *horizontal* edges.

As with the blur, each kernel is separable. Each filter is therefore computed as two 1×3 passes: a [1 2 1] smoothing pass along one direction, then a [-1 0 1] difference pass along the other. Dividing by 4 keeps the results between -255 and 255. Because the results can be negative, they are stored in a signed 16-bit image (CV_16SC3) rather than an 8-bit one. For display, the absolute value is taken and converted back to 8 bits, so both positive and negative edges appear bright. At the image border, missing neighbours are replaced by the nearest valid row or column.

Example outputs are shown with the gradient magnitude in Figure 7.

Task 8: Gradient magnitude

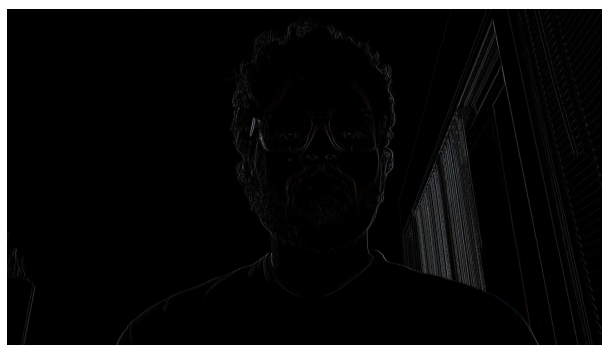
Pressing `m` shows the gradient magnitude. It combines the Sobel X and Y results into a single edge image that responds to edges in every direction. For each pixel and colour channel, the magnitude is the Euclidean length of the gradient vector:

$$M = \sqrt{S_X^2 + S_Y^2}$$

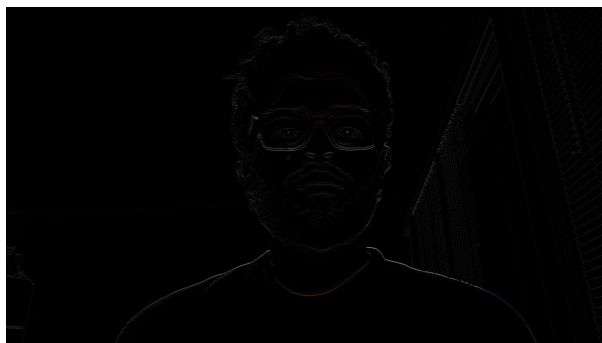
The result is capped at 255 and stored as an ordinary 8-bit colour image, so it can be displayed directly. Squaring removes the sign, so the direction of the brightness change no longer matters, only its strength. Edges at any angle, including diagonals, show up with similar brightness. Because each channel is processed separately, an edge between two colours of similar brightness still shows up in the channels where they differ, which gives the output its coloured edges.



(a) Original



(b) |Sobel X|: vertical edges



(c) |Sobel Y|: horizontal edges



(d) Gradient magnitude

Figure 7: The same scene with the Sobel X and Sobel Y filters and the gradient magnitude. Sobel X highlights only vertical edges and Sobel Y only horizontal ones. The magnitude combines them to show edges in all directions.

Task 9: Blur and quantize

Pressing `1` turns the video into a flat, poster-like cartoon. First the frame is blurred with the separable 5×5 Gaussian from Task 6. Then each colour channel is *quantized* into a fixed number of levels. The live video uses 10 levels. With L levels the bucket size is $b = \lfloor 255/L \rfloor$, and each value x is replaced by the bottom of its bucket:

$$x' = \left\lfloor \frac{x}{b} \right\rfloor b$$

For $L = 10$, $b = 25$, so each channel can take only the values $0, 25, 50, \dots, 225$. The image can then contain at most $10^3 = 1000$ distinct colours, instead of about 16.7 million. Because b is rounded down, values above 250 would otherwise land in an eleventh bucket, so they are merged into the top one.

Blurring first matters. It removes fine texture and camera noise, so nearby pixels fall into the same bucket. The result is large flat regions of colour with clean boundaries rather than speckled noise.

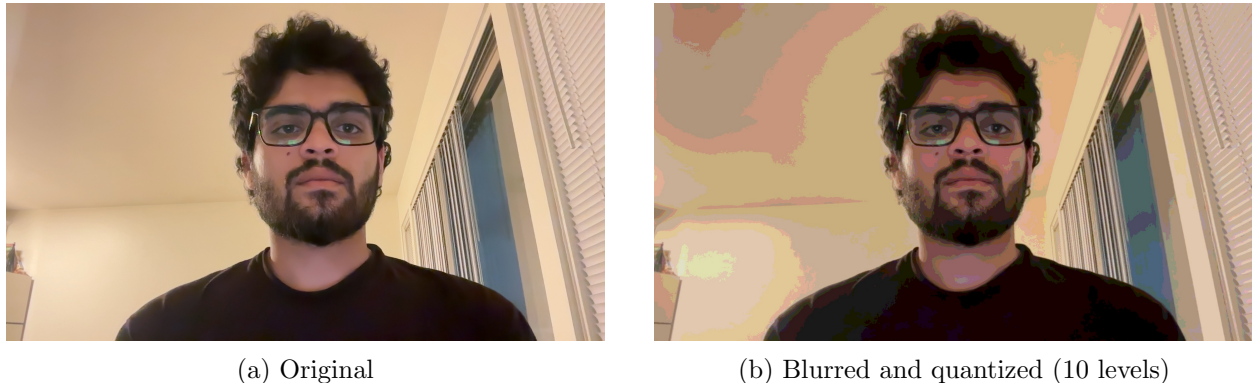


Figure 8: Blurring followed by quantization to 10 levels per channel. Smooth gradients such as shading and skin tones become visible bands of flat colour.

Task 10: Face detection

Pressing **f** toggles face detection. It uses the face-detection code provided with the assignment (`faceDetect.cpp`), which applies OpenCV's pre-trained Haar cascade classifier (`haarcascade_frontalface_alt2`). For each frame:

1. The original colour frame is converted to greyscale, because the Haar cascade works on intensity only.
2. The detector halves the image size to save time and equalises its histogram to reduce the effect of lighting. It then scans the image at several scales for the light and dark patterns typical of a frontal face, such as darker eyes and a brighter nose bridge.
3. The detected rectangles are scaled back to full size and drawn on the displayed image. Boxes narrower than 50 pixels are ignored to reduce false detections.

Unlike the other effects, face detection is a separate on/off setting rather than a display mode. Detection always runs on the original frame, but the boxes are drawn on whatever is currently shown. Faces can therefore be highlighted on top of any other effect, for example sepia or the cartoon filter. The Haar detector sometimes misses a face for a frame or two, which makes the box flicker. To prevent this, the last detected boxes are kept for up to 5 frames when no face is found. The provided code also needed one small change to build with OpenCV 5: the Haar cascade class has moved to the `xobjdetect` module, so that header had to be included.

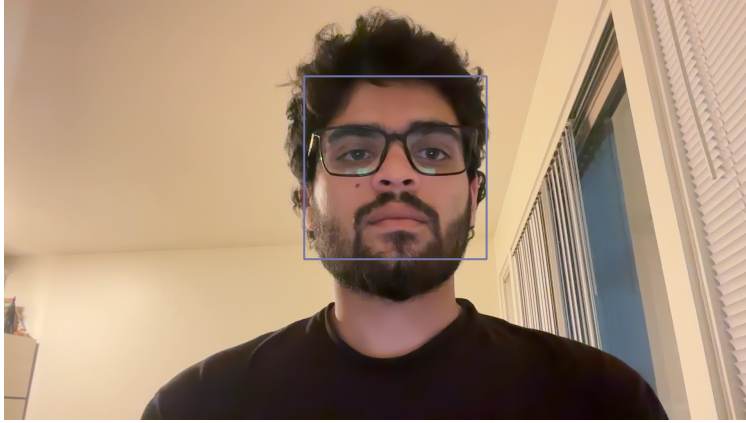


Figure 9: A face detected in the live video stream, marked with a bounding box.

Task 11: Depth estimation and a depth-based filter

This task uses the provided wrapper for *Depth Anything V2* (DA2), a neural network that estimates depth from a single colour image. The network runs through ONNX Runtime with the half-precision model `model_fp16.onnx`. The model is loaded only the first time a depth mode is chosen, so the other effects start instantly.

For each frame:

1. The frame is shrunk to 256 pixels tall before it goes to the network. This keeps the video responsive while still giving a usable depth map.
2. The network outputs a relative depth map, which is resized back to the frame size. It is an 8-bit greyscale image rescaled to $[0, 255]$ for each frame, where 255 is *nearest* and 0 is *farthest*. The values are relative, not metric distances.
3. Each new depth map is averaged with the previous one: $D_t = 0.5 D_{\text{new}} + 0.5 D_{t-1}$. Because the depth is rescaled independently in every frame, the raw output flickers, and this running average makes it much steadier.

Depth view (d). The smoothed depth map is shown with the *inferno* colour map, so near objects appear bright yellow and distant ones dark purple or black.

Creative filter: depth “portrait mode” (z). This filter imitates the shallow depth of field of a portrait-mode phone camera. It keeps the nearby subject sharp and in full colour, while the background is heavily blurred and faded towards grey.

- **Background.** A 5×5 blur has very little effect on its own, so a strong blur is made cheaply instead. The frame is shrunk to a quarter of its size and blurred twice with my `blur5x5.2`, then enlarged back to full size. The blurred image is then mostly desaturated, mixing 40% colour with 60% grey.
- **Blending by depth.** Each pixel’s depth z is mapped to a weight w between 0 and 1 using two thresholds, $lo = 110$ and $hi = 170$. Pixels nearer than hi keep the original sharp pixel ($w = 1$). Pixels farther than lo use the background ($w = 0$). In between, a smoothstep curve $w = t^2(3 - 2t)$ with $t = (z - lo)/(hi - lo)$ gives a soft transition, which avoids a hard cut-out edge around the subject. The output is $w \cdot \text{original} + (1 - w) \cdot \text{background}$.

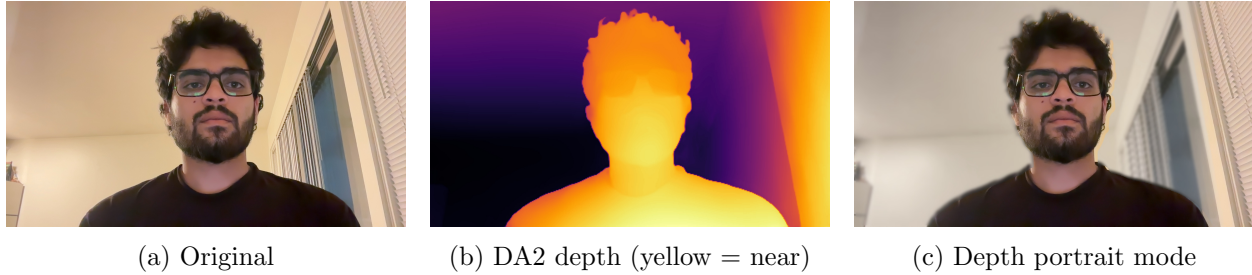


Figure 10: (b) The depth map estimated from the live video stream. (c) The creative depth filter: the nearby subject stays sharp and in colour, while the background is blurred and desaturated.

Task 12: Three more effects

I implemented three further effects. Each one reuses parts of the earlier tasks.

1. Red colour pop (r), a pixel-wise effect. Strongly red regions keep their colour and everything else turns grey, like the familiar “single colour” photo effect. For each pixel, its *redness* is measured as $\rho = R - \max(G, B)$. This is large only for saturated reds; it is small for white, grey, skin highlights, and orange-yellow colours whose green value is also high. The redness is mapped to a weight with a smoothstep between $\rho = 40$ and $\rho = 80$, and the output is $w \cdot \text{original} + (1 - w) \cdot \text{grey}$, where the grey uses the same weights as `cvtColor`. The soft ramp avoids hard, noisy edges around red objects.

2. Cartoon / video abstraction (t). This effect follows Winnemöller, Olsen and Gooch’s real-time video abstraction method. It gives a painted, comic-book look with smooth flat shading and dark outlines. To keep it real-time it runs at half resolution, in four steps:

1. **Edge-preserving smoothing.** The frame is converted to the CIELab colour space and smoothed with four passes of a bilateral filter. Unlike a Gaussian blur, a bilateral filter averages only pixels of similar colour, so it removes texture while keeping strong edges sharp.
2. **Outlines.** A difference of Gaussians (DoG) is computed on the lightness channel after two smoothing passes. It subtracts a wider blur ($\sigma = 1.6$) from a narrower one ($\sigma = 1$). This is near zero in flat areas and dips sharply along edges, which produces the dark cartoon lines.
3. **Soft quantization.** Lightness is snapped towards 10 levels, like Task 9. However, the step between levels is a smooth tanh curve rather than a hard jump. Its sharpness is set per pixel from the lightness gradient, computed with my own Sobel and magnitude functions from Tasks 7–8. Flat regions get soft transitions and textured regions get sharper steps, which reduces flicker between video frames.
4. **Combine.** The quantized lightness is multiplied by the edge map, so outlines darken the image. It is then converted back to BGR and enlarged to full size.

3. Face spotlight (o). This uses the face detector from Task 10. The people in the frame stay in full colour, while the rest of the scene becomes a dimmed greyscale: my custom greyscale from Task 4 at 60% brightness. Rectangular boxes look unnatural, so each face gets an oval spotlight slightly larger than its box, to include hair and chin. The oval has a soft smoothstep edge that

fades into the background. With several faces, each pixel takes the strongest spotlight weight, so every face is lit.



(a) Original



(b) Red colour pop

Figure 11: Red colour pop: only strongly red regions keep their colour.



(a) Original



(b) Cartoon abstraction

Figure 12: Cartoon effect: flat shading from bilateral filtering and soft quantization, with dark DoG outlines.



(a) Original



(b) Face spotlight

Figure 13: Face spotlight: the detected face stays in colour inside a soft oval while the background is dimmed to grey.

3 Extensions

Extension 1: Canny edge detector from scratch (e)

Canny is the standard edge detector. Where the gradient magnitude of Task 8 gives thick, blurry edges and responds to every bit of texture, Canny produces thin, connected, one-pixel lines along the real outlines. I wrote it from scratch in `canny()` in `filter.cpp`, reusing my own filters for the first steps. No OpenCV image-processing functions are used. Pressing `e` in the video toggles it, and it works together with the face boxes (`f`).

The filter runs in six steps:

1. **Blur and greyscale.** The frame is smoothed with my separable 5×5 Gaussian (`blur5x5.2`, Task 6) so that noise does not turn into fake edges, then converted to grey with the same weights as `cvtColor`.
2. **Sobel X and Y.** My 3×3 Sobel filters from Task 7 give the horizontal and vertical gradients g_x and g_y .
3. **Magnitude and direction.** The edge strength is $\sqrt{g_x^2 + g_y^2}$ as in Task 8, and the gradient direction is $\theta = \text{atan2}(g_y, g_x)$. The direction is rounded to one of four sectors: 0° , 45° , 90° or 135° .
4. **Non-maximum suppression (thinning).** A pixel is kept only if its magnitude is at least as large as both neighbours *across* the edge, i.e. along the gradient direction. This turns thick ridges into one-pixel lines.
5. **Double threshold.** Thinned pixels with magnitude $\geq \text{high}$ are *strong* edges, those between `low` and `high` are *weak*, and the rest are discarded.
6. **Hysteresis.** Starting from every strong pixel, a stack is used to grow into touching weak pixels (8-neighbourhood), which then become strong too. Weak pixels that never connect to a strong edge are dropped. This keeps long edges unbroken where they fade, while removing isolated noise.

The thresholds are in the same units as the Task 8 magnitude (about 0–361). The video uses `low = 5` and `high = 50`: the high value decides where edges start, and the low value lets them continue through weaker parts, such as the nose and the frame of the glasses. To check the result, I ran OpenCV’s built-in `cv::Canny` on the same image with matching settings. Because my Sobel divides by 4, its thresholds are multiplied by 4. The two outputs are almost identical (Figure 14). With `-02`, my version takes about 16 ms per 1280×720 frame, fast enough for live video.

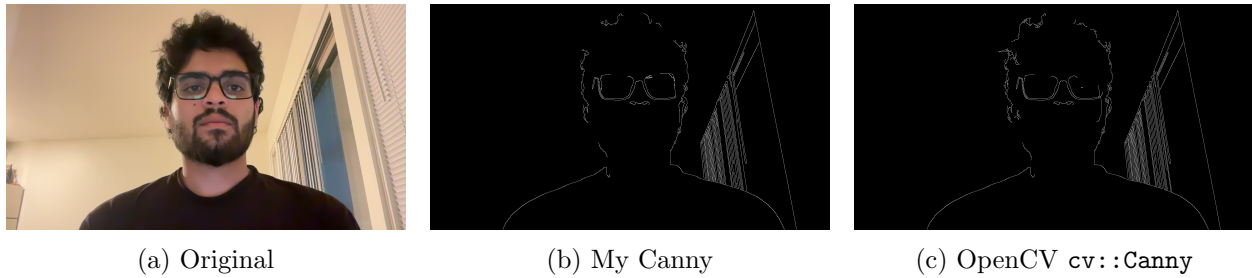


Figure 14: Extension 1: my from-scratch Canny edge detector (`low = 5`, `high = 50`) compared with OpenCV's built-in version using matching settings.

I also tried running Canny without the blur in step 1 and raising `low` to 5 (Figure 15). Skipping the blur keeps fine detail that the Gaussian would smooth away, so the hair outline and glasses come out sharper and more detailed. On its own, however, it also turns hair and beard texture into many small noisy edges. The higher `low` threshold removes most of these weak texture edges while the strong outlines stay connected. The trade-off is that soft facial features such as the nose and mouth become fainter.



Figure 15: Canny without blur, `low = 5`, `high = 50`, cropped to the face.

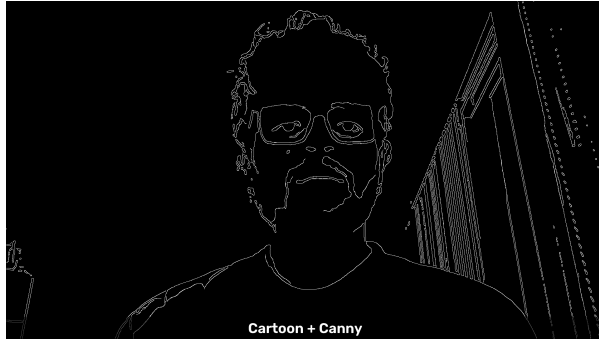
Extension 2: Stacking filters (`chainDisplay`)

In `vidDisplay` only one effect is active at a time, and it is always applied to the original camera frame. The new program `chainDisplay.cpp` lets the user *stack* filters: each filter key adds that filter to the end of a chain, so any filters can be combined in any order, including the same filter several times. `u` removes the last filter, `c` clears the chain, and `s` saves the current picture. The filters

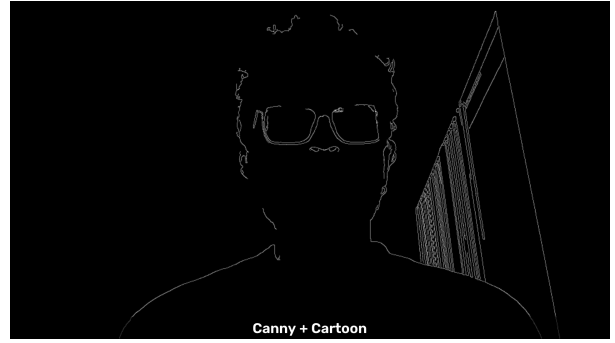
are not rewritten. The program includes `filter.h` and calls the same functions from `filter.cpp` used by `vidDisplay`.

For every frame, the chain starts again from the original picture, and each filter is applied to the *output of the previous one*. With **Sepia + Blur + Canny**, for example, the blur works on the sepia image and Canny works on the blurred sepia image. To make every combination possible, each step leaves the picture as a normal 8-bit 3-channel image. The OpenCV grey output is converted back to 3 channels, and the signed Sobel outputs are converted with `convertScaleAbs`. Face detection and depth are always computed from the *original* frame, because both work badly on an already filtered picture, but the spotlight and depth blur are applied to the filtered result. The current chain is written at the bottom of the video on a dark strip, e.g. “Sepia + Blur + Face Spotlight”. The text shrinks automatically for long chains.

Figure 16 shows some combinations. The first two use the same two filters in opposite orders and look completely different, which shows that the order of filters matters. Cartoon followed by Canny finds the edges between the flat cartoon regions, which gives a detailed sketch of the face and beard. Canny followed by Cartoon only smooths an image that is already just thin lines, so little changes. Applying Cartoon three times makes the colour regions flatter and the outlines darker each time. Depth Focus followed by Blur Quantize blurs the background first, so it quantizes into large, smooth colour areas while the face keeps its detail. Colour Pop followed by Cartoon gives a grey cartoon, since there is no strong red in this scene. Sepia with three blurs and a face spotlight gives a soft old-photo portrait.



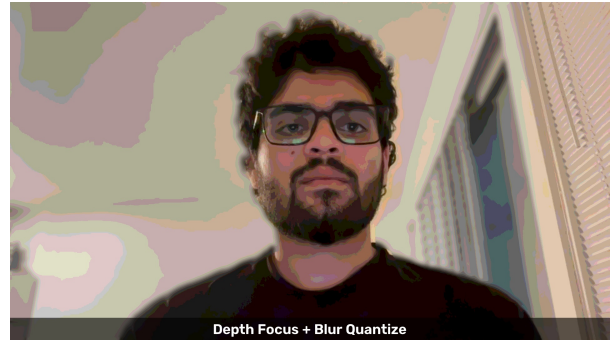
(a) Cartoon + Canny



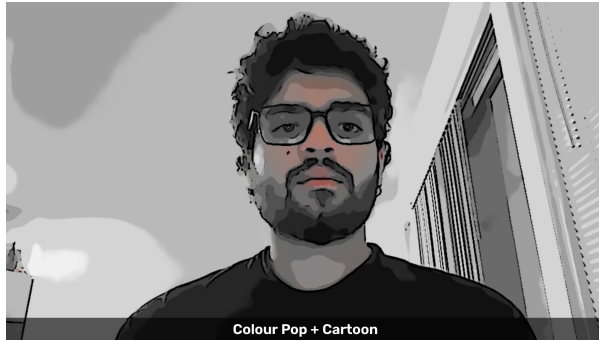
(b) Canny + Cartoon



(c) Cartoon + Cartoon + Cartoon



(d) Depth Focus + Blur Quantize



(e) Colour Pop + Cartoon



(f) Sepia + Blur ($\times 3$) + Face Spotlight

Figure 16: Extension 2: stacked filters. Each filter is applied to the output of the previous one, and the chain is shown at the bottom of the video. (a) and (b) use the same filters in opposite orders.

4 Reflection

Setting up C++ with OpenCV on macOS clarified the build, link, and real-time capture steps, along with OpenCV's BGR layout. Implementing filters directly reinforced that images are typed matrix grids—for example, signed 16-bit Sobel outputs must be converted back to 8-bit for display.

Optimizing the 5×5 Gaussian blur by separating it into two 1×5 passes with row pointers dramatically improved throughput. Compiler flags proved equally critical: compiling with `-O2` yielded a $10\times$ – $30\times$ speedup required for smooth real-time performance.

Working with face detection and depth estimation highlighted real-world visual noise. Temporal

frame-holding for faces and depth averaging over time were essential to eliminate flickering. Writing Canny edge detection from scratch showed that it detects raw brightness edges rather than semantic boundaries, where removing blur adds unwanted texture noise.

Finally, building stacked filter chains in `chainDisplay` demonstrated that image operations are non-commutative—changing the order produces completely different results. Writing standalone filter functions emphasized the value of modular, reusable code design.

5 Acknowledgements

- Youtube.
- Prof. Bruce Maxwell’s provided face-detection code (`faceDetect.cpp`, `faceDetect.h`, `showFaces.cpp`) and the OpenCV Haar cascade file `haarcascade_frontalface_alt2.xml`.
- The provided Depth Anything V2 wrapper (`DA2Network.hpp`) and ONNX model; L. Yang *et al.*, “Depth Anything V2,” 2024. Inference runs with ONNX Runtime.
- H. Winnemöller, S. C. Olsen, and B. Gooch, “Real-Time Video Abstraction,” *ACM Transactions on Graphics (SIGGRAPH)*, 25(3), 2006. This was the basis for the cartoon effect.
- OpenCV documentation (<https://docs.opencv.org>), in particular the pages for `VideoCapture`, `imshow`, `waitKey`, and `cvtColor`.

LLM Usage Statement. I used Claude as an interactive learning and development aid throughout this project. I designed the core architecture, algorithm selection, and logic for each image filter, leveraging YouTube tutorials and course materials for foundational concepts. Claude was utilized to assist with drafting report formatting in $\text{\LaTeX}$, writing initial code boilerplate, and debugging implementation issues. All generated code was reviewed, adapted, and integrated by me to ensure correctness and alignment with project requirements.